

1. What is Serilog?

Serilog is a **structured logging framework** for .NET.

Unlike traditional logging (plain text messages), Serilog logs **structured data** (key–value pairs), making logs:

- Machine-readable
- Easy to search, filter, and analyze
- Ideal for centralized logging systems (Seq, ELK, Azure Monitor, etc.)

Key Characteristics

- Structured logging (JSON-based)
 - Multiple sinks (Console, File, Seq, SQL Server, Elasticsearch)
 - Enrichment support (RequestId, MachineName, Environment)
 - Excellent integration with ASP.NET Core
-

2. Why Use Serilog in ASP.NET Core REST APIs?

In REST APIs, logging is critical for:

- Request/response tracing
- Debugging production issues
- Auditing and compliance
- Observability in microservices

Serilog advantages over default ILogger

Feature	ILogger	Serilog
Structured logs	Limited	Yes
Centralized logging	No	Yes
Rich sinks	No	Yes
Log enrichment	Basic	Advanced

3. Logging Levels in Serilog

Level	Usage
Verbose	Very detailed diagnostics
Debug	Development debugging
Information	Normal application flow
Warning	Unexpected but recoverable
Error	Failure in execution
Fatal	Application crash

4. Install Required NuGet Packages

In your **ASP.NET Core Web API**, install:

```
dotnet add package Serilog.AspNetCore
```

```
dotnet add package Serilog.Sinks.Console
```

```
dotnet add package Serilog.Sinks.File
```

(Optional, for centralized logging later)

```
dotnet add package Serilog.Sinks.Seq
```

5. Configure Serilog in ASP.NET Core (Program.cs)

Program.cs (Minimal Hosting Model – .NET 6+)

```
using Serilog;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Configure Serilog
```

```
Log.Logger = new LoggerConfiguration()
```

```
.MinimumLevel.Information()

.Enrich.FromLogContext()

.WriteTo.Console()

.WriteTo.File(
    path: "Logs/log-.txt",
    rollingInterval: RollingInterval.Day)

.CreateLogger();

// Replace default logger with Serilog
builder.Host.UseSerilog();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSerilogRequestLogging(); // Logs HTTP requests

app.UseSwagger();
app.UseSwaggerUI();

app.UseAuthorization();
app.MapControllers();

app.Run();
```

What this does

- Replaces Microsoft logging with Serilog
 - Logs requests automatically
 - Writes logs to:
 - Console
 - Rolling log files (daily)
-

6. Logging in Controllers (Practical Example)

Sample Controller

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.Extensions.Logging;
```

```
[ApiController]
```

```
[Route("api/products")]
```

```
public class ProductsController : ControllerBase
```

```
{
```

```
    private readonly ILogger<ProductsController> _logger;
```

```
    public ProductsController(ILogger<ProductsController> logger)
```

```
    {
```

```
        _logger = logger;
```

```
    }
```

```
    [HttpGet("{id}")]
```

```
    public IActionResult GetProduct(int id)
```

```
    {
```

```

_logger.LogInformation("Fetching product with Id {ProductId}", id);

if (id <= 0)
{
    _logger.LogWarning("Invalid product id received: {ProductId}", id);
    return BadRequest();
}

try
{
    var product = new { Id = id, Name = "Laptop", Price = 50000 };
    _logger.LogInformation("Product retrieved successfully {@Product}", product);

    return Ok(product);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Error occurred while fetching product {ProductId}", id);
    return StatusCode(500);
}
}

```

Why this is powerful

- {ProductId} is a **structured property**
- { @Product } logs the entire object
- Logs are searchable in centralized systems

7. Sample Log Output (Structured)

Console / File

[09:10:25 INF] Fetching product with Id 10

[09:10:25 INF] Product retrieved successfully { Id: 10, Name: Laptop, Price: 50000 }

Internally Stored as JSON

```
{  
  "Timestamp": "2025-12-15T09:10:25",  
  "Level": "Information",  
  "MessageTemplate": "Fetching product with Id {ProductId}",  
  "Properties": {  
    "ProductId": 10  
  }  
}
```

8. Automatic HTTP Request Logging

This line enables **request-level logs**:

```
app.UseSerilogRequestLogging();
```

Logs:

- HTTP Method
- Path
- Status code
- Execution time

Example:

HTTP GET /api/products/10 responded 200 in 45 ms

9. Log Enrichment (Optional but Recommended)

`.Enrich.WithMachineName()`

`.Enrich.WithEnvironmentName()`

Adds:

- MachineName
- Environment (Development / Production)

Useful in microservices and cloud deployments.

10. Where Serilog Fits in Clean Architecture

Given your **Clean Architecture and Microservices background**:

- **API Layer** → Configure Serilog
- **Application Layer** → Use `ILogger<T>`
- **Infrastructure Layer** → No logging framework dependency
- **Centralized logging** → Seq / ELK via sinks

Serilog remains an **outer-layer concern**, respecting architectural boundaries.

11. Summary

- Serilog is a **production-grade structured logging framework**
- Easy integration with ASP.NET Core
- Supports centralized logging and observability(What is happening inside my system right now)
- Essential for microservices and distributed systems
- Replaces default logging seamlessly